

Expense Tracker

Group Project Report

Course: CST8219 – C++ Programming

Project Title: Expense Tracker – Personal Finance & Budgeting Desktop Application

Student Information

Student Name	Student ID
Leo Paquette	041147786
Jacob Paulin	041024610
Wesley Deir	041162206

August 16, 2026

1. Introduction

The Expense Tracker is a personal finance application and budget management system, written in C++ and using QT Widgets for the front end. It lets users record their income and expenses, organise them by category, and review their spending and budgets. The target user is someone who wants a local expense tracker with no account registration.

2. Project Objectives

Including the main goal to build a functional expense tracker, the project was used to evaluate our ability to apply C++ concepts:

- Object oriented design with a Transaction abstract class and Income/Expense classes that derive it
- Inheritance and runtime polymorphism with virtual functions (computeImpact and displayTransaction)
- Class templates and template member functions (AnalyticsEngine<T>, TransactionManager::addTransaction<T>, TransactionManager::editTransaction<T>)
- Operator overloading (operator<< and operator==) and friend functions
- File I/O through JSON serialization
- GUI built with QT Widgets, including forms, tabs and STL containers

3. System Overview

The system is built from five main components: the Transaction hierarchy (Transaction/Expense/Income) and Category as the data model, TransactionManager for the business logic, DataManager for JSON persistence, AnalyticsEngine for statistics calculations, and GUI as the QT front end.

4. System Architecture

The project uses a layered architecture and keeps QT separate from the reusable business logic:

- **Data/Model Layer:** Transaction abstract class with Expense and Income derived classes, with Category. Each validates its own field in the constructor/setters and serializes with toJSON()/fromJSON().
- **Business Logic Layer:** TransactionManager owns every instance of Transaction and Category, and exposes add/edit/delete/search/filter operations. Only source the GUI interacts with.
- **Persistence Layer:** DataManager read and writes two JSON files, using toJSON()/fromJSON() methods.
- **Analytics Layer:** AnalyticsEngine<T> template computes totals, savings, and category budget usage from the results of a TransactionManager query.
- **Presentation Layer:** GUI owns one TransactionManager and one AnalyticsEngine<>, wires actions from GUI.ui to slots, and translates results to table rows and message boxes.

5. Class Design

Transaction (abstract)

- **Purpose:** Base class representing a single financial record shared by expenses and income.
- **Key Attributes:** transactionID, date, amount, category, description (protected)
- **Key Methods:** computeImpact() – pure virtual; getters/setters (validated); displayTransaction() – virtual; toJSON()/fromJSON(); operator==; operator<< (friend)
- **Relationships:** Base class of Expense and Income (inheritance); owned by TransactionManager via std::unique_ptr (composition)

Expense

- **Purpose:** A transaction that reduces the user's budget.
- **Key Attributes:** paymentMethod (in addition to the inherited fields)
- **Key Methods:** computeImpact() overridden to return true; displayTransaction() override; toJSON()/fromJSON()
- **Relationships:** Inherits Transaction

Income

- **Purpose:** A transaction that adds to the user's budget.
- **Key Attributes:** source (in addition to the inherited fields)
- **Key Methods:** computeImpact() overridden to return false; displayTransaction() override; toJSON()/fromJSON()
- **Relationships:** Inherits Transaction

Category

- **Purpose:** Represents a spending bucket with a monthly budget.
- **Key Attributes:** categoryID, name, monthlyBudget
- **Key Methods:** updateBudget(), displayCategorySummary(), toJSON()/fromJSON(), operator==, operator<< (friend)
- **Relationships:** Referenced by name from Transaction::category (logical association, not a pointer/reference)

TransactionManager

- **Purpose:** Central in-memory store and business-logic layer for transactions and categories.
- **Key Attributes:** std::vector<std::unique_ptr<Transaction>> transactions; std::vector<std::unique_ptr<Category>> categories
- **Key Methods:** addTransaction<T>()/editTransaction<T>() (templates), deleteTransaction(), addCategory()/editCategory()/deleteCategory(), searchTransactions(), filterByCategory(), filterByDateRange(), load()/save()
- **Relationships:** Composition over Transaction and Category (owns them via unique_ptr); depends on DataManager for persistence

DataManager

- **Purpose:** Reads and writes the transactions/categories JSON files.
- **Key Attributes:** transactionFilePath, categoriesFilePath (const)
- **Key Methods:** loadData(), saveData()
- **Relationships:** Used by TransactionManager (dependency); depends on Transaction::fromJson/toJSON and Category::fromJson/toJSON

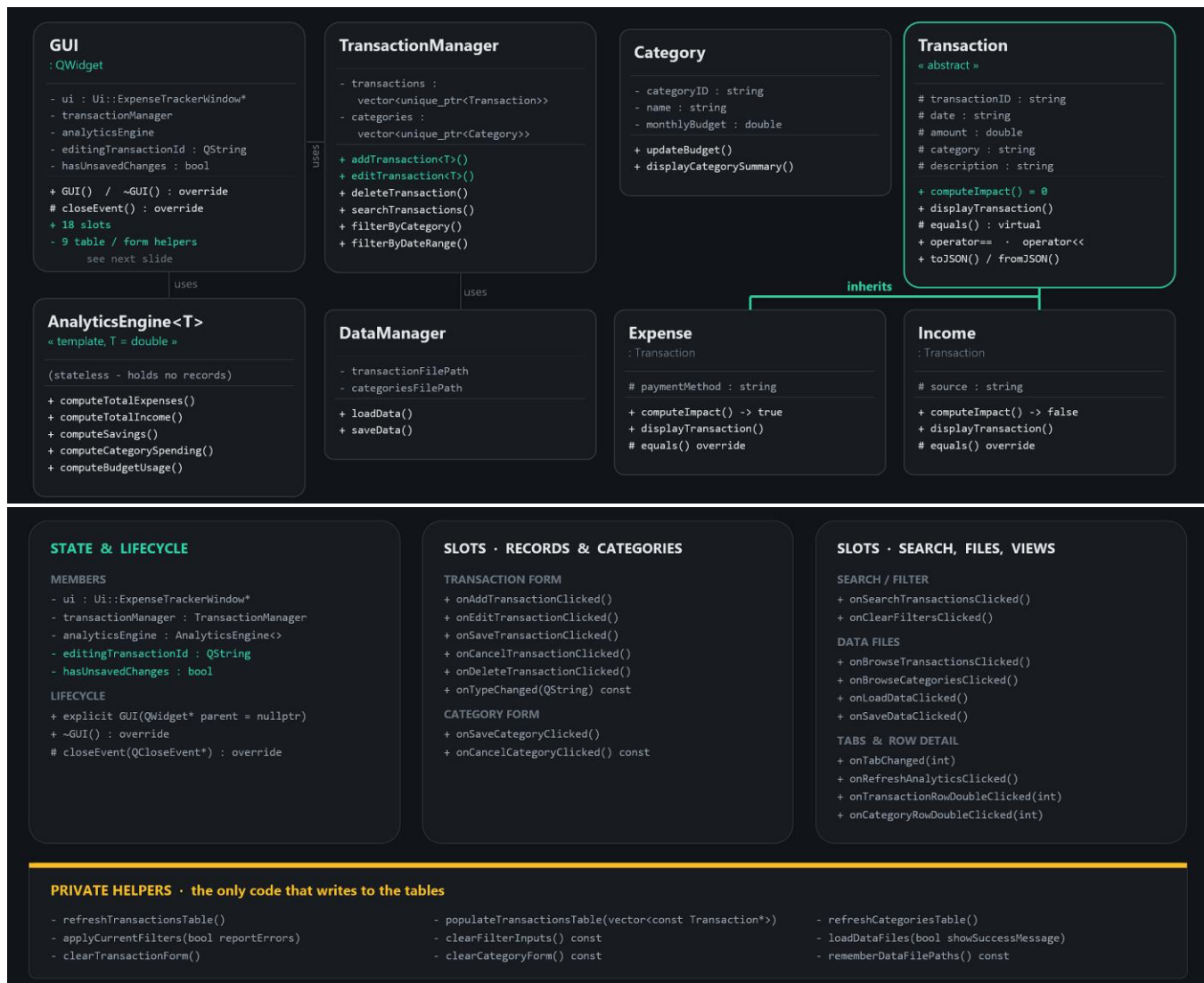
AnalyticsEngine<T = double>

- **Purpose:** Generic template class that computes financial summaries (expenses, income, savings, per-category spending, budget usage) from a set of transactions. T is the arithmetic type totals accumulate in and defaults to double. Header-only.
- **Key Attributes:** None because its stateless
- **Nested Types:** BudgetUsage struct - categoryName, budget, spent, usagePercent - one row per category for the analytics table.
- **Key Methods:** computeTotalExpenses(), computeTotalIncome(), computeSavings(), computeCategorySpending(), computeBudgetUsage(); private helpers spentIn() and percentOf().
- **Relationships:** Operates on const Transaction* / const Category* supplied by TransactionManager; uses util::str_to_lower so category grouping is case insensitive; consumed by the GUI's analytics tab.

GUI

- **Purpose:** Qt Widgets main window; the only class in the project aware of Qt. Hosts the main, transaction-form, category and analytics tabs.
- **Key Attributes:** ui (the Designer widget tree), transactionManager, analyticsEngine, plus editingTransactionId (empty = adding, set = editing) and hasUnsavedChanges.
- **Key Methods:** slot handlers per control. closeEvent() override prompting Save/Discard/Cancel on unsaved changes; private table-population and form-reset helpers (applyCurrentFilters, populateTransactionsTable, refreshCategoriesTable, clearTransactionForm, loadDataFiles, rememberDataFilePaths).
- **Relationships:** Inherits QWidget; owns a TransactionManager and an AnalyticsEngine<> (composition); remembers the last-used data file paths via QSettings so startup reloads them.

6. UML Class Diagram - taken directly from our presentation



7. Implementation Details

7.1 Object-Oriented Programming

Classes and Objects

Transaction, Expense, Income, Category, TransactionManager, DataManager, AnalyticsEngine and GUI are all single purpose, distinct classes.

Encapsulation

Fields are protected or private and only mutable through validating setters that throw `std::invalid_argument` on invalid input. No object can be pushed into an invalid state.

Inheritance

Expense and Income are both derived from the Transaction abstract class. They inherit all Transaction attributes while including their own specific field.

Polymorphism

computeImpact() and displayTransaction() are virtual. GUI::populateTransactionsTable only uses dynamic_cast when retrieving the derived class field. AnalyticsEngine calls computeImpact().

Abstract Classes

Transaction is an abstract class through its pure virtual function computeImpact(), it can only be instantiated by Expense or Income.

7.2 Templates and Generic Programming

AnalyticsEngine<T = double> is a class template so the same logic could be reused with a different numeric type. TransactionManager::addTransaction<T>() and TransactionManager::editTransaction<T>() are template member functions constrained at compile time so only Transaction derived types can be passed.

7.3 Operator Overloading

Two operators are overloaded per class in the data model. operator== compares two Transaction or Category objects field by field and returns true only if every field matches. For Transaction, equality checks that operands share the same concrete runtime type via typeid. An Expense and an Income that happen to have identical id/date/amount/category/description are never considered equal, because equals() is virtual and each subclass extends it with its own field (paymentMethod for Expense, source for Income). Category equality compares categoryID, name (case insensitive) and monthlyBudget. operator<< is overloaded for both transaction and category so any object can be written straight to an ostream.

7.4 Friend Functions

operator<< cannot be a member function without changing the calling convention, but it still needs direct access to Transaction's protected fields and Category's private fields. Both classes declare friend ostream& operator<<(ostream&, const Transaction&/Category&) so the free function can format every field directly instead of going through getters.

7.5 Data Structures (STL)

- **std::vector<std::unique_ptr<T>>**: Holds the manager's owned transactions and categories, contiguous storage with exclusive ownership.
- **std::map<std::string, T>**: Aggregates spending per category in AnalyticsEngine, chosen for key ordering and lookup by category name.
- **std::optional<T>**: Models filters that can be supplied to searchTransactions/filterByDateRange.
- **std::expected<T, E>**: Used for defined failure modes instead of throwing sentinel values
- **std::regex**: Validates ID and date formats

7.6 Memory Management

Every Transaction and Category is owned through TransactionManager's vectors using `std::unique_ptr`. They are freed automatically when the manager or the vectors are destroyed. Meaning there is no manual new/delete and no possibility of a double delete. The only manual memory management is done in GUI and is required. It allocates the Designer generated `UI::ExpenseTrackerWindow` with new in the constructor and delete in its destructor. All model layer classes follow RAI: validation happens in the constructor, so an object can only exist if it's fully valid. Otherwise an exception is thrown and no partially formed object exists.

8. Graphical User Interface (GUI)

Our base layer is a QWidget subclass called GUI. Everything sits inside a QTabWidget with four tabs.

Main: A table in the centre showing ID, Date, Amount, Category, Description, Type and Method/Source, generated from the JSON. Above it, three buttons for Add/Edit/Delete Transaction and a Search/Filter card holding a date edit, two text fields, and Search and Clear Filters buttons. Below that, the Save/Load section: two text fields for the file paths, a Browse button for each so users can pick data files out of their file explorer and Save Data and Load Data buttons. Whichever paths were used last are remembered, so relaunching the app brings you back to the files you were working on.

Transaction Form: A plain form with the fields needed to create or edit a transaction (text inputs for most, dropdowns where the value is a fixed choice) plus Save and Cancel at the bottom. One extra field sits alongside them: a spending threshold the user sets themselves. Save an expense above it and the app asks "This expense exceeds \$1000. Are you sure you want to continue?" before the record lands in the table, and nothing is stored unless the user says yes. Income is exempt.

Category/Budget Form: The same form treatment for categories, with a table underneath listing each category and its monthly budget.

Analytics Tab: Three cards across the top - Total Income, Total Expenses, and Savings, which recolours itself based on whether the computed total is positive. A Refresh button below them updates the cards and the table. The table shows each category, its budget, what was spent against it, and the percentage used. That last column is a run-time populated usage bar that colours itself off constants declared at the top of `gui.cpp`: it runs green while a category is comfortable, turns amber once spending passes 80% of the budget, and turns red once it passes 100%, so an overspent category is obvious without reading the numbers.

Input validation: bad data is caught on the way in from the JSON files and at every form field: wrong types, malformed IDs (TRX-xxxxxx where TRX-xxxx was wanted), dates in the future, transactions filed under a category that doesn't exist. Every test and edge case in the spec is accounted for, and each one shows the user a warning naming what went wrong and how to fix it.

Enhanced validation features: The spec asks for at least three of the six, and we implemented four: Features 2, 3, 5 and 6. Feature 2, duplicate detection, is handled by TransactionManager refusing the record and the GUI reporting it - "Transaction TRX-0001 already exists." when an ID is already taken, and "A category named "Food" already exists." when a category name is reused, which is matched case insensitively so Food and food are never

both created. Feature 3, the budget warning system, is the usage bar described above, warning at over 80% and again at over 100%. Feature 5, unsaved changes detection, is a `closeEvent` override that asks "You have unsaved changes. Do you want to save before exiting?" with Save, Discard and Cancel; choosing Save on a read-only or locked file leaves the window open rather than closing it and losing the work. Feature 6, the spending alert, is the configurable per-transaction threshold on the Transaction Form.

9. Data Persistence

Data is stored in JSON files, one for transactions and one for categories. Users select a file to load from in their system path. Serialization is handled per class, using `toJSON()/fromJson()`, with Transaction having a type (1 for Expense, 2 for Income) to correctly dispatch the correct derived class. `DataManager::loadData()` loads the data from the files into their own vector, and `DataManager::saveData()` saves the data stored in the vectors to the JSON files.

10. Input Validation and Error Handling

Input validation primarily occurs at the GUI level, and typically sees data being passed down from there already being sanitized and validated. However, throughout all parts of the application standard practices of defensive programming is observed to further enhance validation of various levels of input.

Error handling at the levels below the GUI aim to be as verbose as can be, often using the pattern of a nested enum class of possible errors to be returned via `std::expected` or `std::optional` to indicate error cases. In situations where a return value is not possible such as in constructors, we then resort to throwing errors. We also fall back on throwing errors in last resort situations, such as in the TransactionManager's `editTransaction` message.

Because Transaction is an abstract class, the method needs to be generic and parse the implemented type by casting to a finite number of possible classes. Specifically, the classes Expense or Income, and in the event, we cannot cast to either of those 2 types it means 1 of 2 things. Either a 3rd implementation of Transaction was made and this method's code was not updated, or somehow somehow there was a non-transaction type passed to the method. Both of these situations are not only developer error, but fatal and unrecoverable which justify their use of throwing an error.

11. Testing

All fifteen edge cases below were run against the finished application on 2026-08-06 and all of them passed.

GUI edge cases

Pressing Save several times in a row on the transaction form created one record, not several. With a message box open, clicking through to the Analytics tab did nothing — the dialog held focus until it was dismissed. Closing the window with unsaved work prompted "You have unsaved changes. Do you want to save before exiting?" rather than discarding it. Pressing Edit with no row selected answered "Select a transaction in the table first.", and Delete with no row selected did the same.

Data edge cases

A file of several thousand transactions loaded cleanly and reported "Loaded 5006 records." An amount above the ceiling was refused with "Invalid amount. Must be a number between 0 and 1,000,000." Descriptions were tested at both ends: 100 characters was accepted, 650 was rejected with "Invalid description. Must be 5-100 characters and contain a letter." Data files containing duplicate IDs were refused as a whole — "The data files contain duplicate IDs, so nothing was loaded." — so a partial load could not leave the application half populated. A category with no budget was caught by "Invalid monthly budget. Must be a number between 0 and 1,000,000."

File edge cases

Pointing the application at a file that isn't there produced "The transactions file could not be found." An empty file, a corrupted JSON file, and a file with invalid character encoding were each loaded in turn, and none of them crashed the application. Saving over a read-only file reported "The file couldn't be opened."; both target files were byte-identical afterwards, so nothing was lost or partially written.

12. Challenges and Solutions

One of the challenges we faced was while we did do some unit tests against the initial design before the GUI was hooked up, it was hard to get the test to cover 100% of the design as it was hard to predict 100% how the GUI would tie into the rest of the code. Additionally, it was also hard to predict how the rest of the code would change because of the hooking up of the GUI.

As the GUI was getting hooked up the rest of the edge cases surfaced and it was easy to handle them as a team because we made sure to communicate with each other when issues are encountered. We also made sure to leave the fixes to the respective team member who was responsible for that section of the code, this made fixes quick and easy to implement since everyone worried about their part and knew their part intimately.

13. Future Improvements

One of the future improvements we would make to the application is the creation of a mobile version of the application, and likely after that, adding syncing of data across desktop and mobile versions of the application. A better persistence model would be nice too so we're not limited to only passing json files back and forth to the application.

14. Conclusion

This Expense Tracker meets the criteria assigned to us in the project objectives: a QT Widgets application that records user income and expenses, organises them into categories with budgets, and persist the data via JSON. Building this application allowed us to exercise the learning material from the course — an abstract Transaction hierarchy with virtual dispatch, a template AnalyticsEngine with template functions, operator overloading with friend function, RAII based memory management via `std::unique_ptr`, and modern error handling via

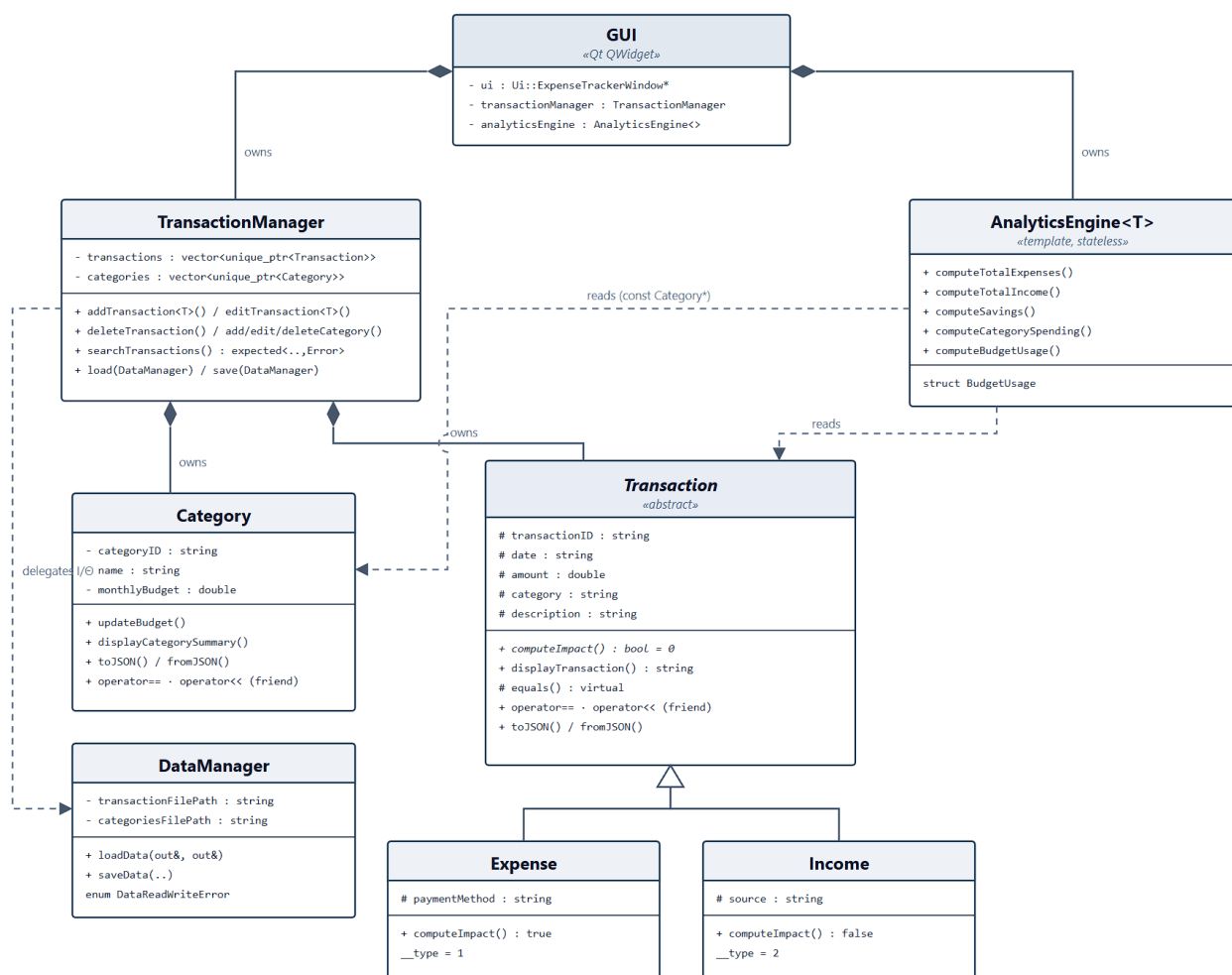
std::optional and std::expected. The result is a small but complete personal finance tool with future iterations including a mobile version.

15. References

<https://doc.qt.io/qt-6/> <https://github.com/nlohmann/json> <https://doc.qt.io/qt-6/qpalette.html>
<https://doc.qt.io/qt-6/qstylefactory.html> <https://doc.qt.io/qt-6/resources.html> <https://doc.qt.io/qt-6/stylesheets-reference.html> <https://doc.qt.io/qt-6/stylesheets-examples.html> <https://doc.qt.io/qt-6/stylesheets-syntax.html> <https://doc.qt.io/qt-6/appicon.html> <https://stackoverflow.com/questions/15686501/how-to-resize-qtablewidget-columns> <https://forum.qt.io/topic/139637/how-to-create-a-button-and-place-it-where-i-want-and-then-run-a-function-when-the-button-is-clicked-c-qt/3>
<https://github.com/ColinDuquesnoy/QDarkStyleSheet>

16. Appendix

Appendix A – Class Relationship Diagram



Appendix B – Code Snippets

1. ABSTRACT CLASS WITH PURE VIRTUAL FUNCTION

include/Transaction.h, lines 73-74

```
// Computes whether this transaction reduces the budget; overridden per
transaction type
virtual bool computeImpact() const = 0;
```

2. INHERITANCE AND POLYMORPHISM

src/Expense.cpp, lines 38-41

```
// Expenses always reduce the budget
bool Expense::computeImpact() const{
    return true;
}
```

src/Income.cpp, lines 40-43

```
// Income never reduces the budget
bool Income::computeImpact() const {
    return false;
}
```

3. CONSTRUCTOR CHAINING

src/Expense.cpp, lines 13-20

```
// Constructor validating the payment method before delegating to Transaction
Expense::Expense(const string &id, const string &date, double amount, const string
&category, const string &description, const string &paymentMethod)
    :Transaction(id, date, amount, category, description) {
    if (!isValidPaymentMethod(paymentMethod)) {
        throw invalid_argument("Invalid payment method");
    }
}
```

```
    this->paymentMethod = paymentMethod;
}
```

4. FRIEND FUNCTION

include/Transaction.h, line 82

```
    friend ostream& operator<<(ostream& os, const Transaction& t);
```

src/Transaction.cpp, lines 214-222

```
// Friend function writing a transaction's fields to the given stream
ostream& operator<<(ostream& os, const Transaction& t) {
    os << "ID: " << t.transactionID
        << ", Date: " << t.date
        << ", Amount: " << t.amount
        << ", Category: " << t.category
        << ", Description: " << t.description;
    return os;
}
```

5. OPERATOR OVERLOADING (second operator)

src/Transaction.cpp, lines 198-202

```
// Equality requires both sides to be the same concrete type before comparing any
// fields, so an
// Expense and an Income with identical base fields are never reported equal
bool Transaction::operator==(const Transaction& other) const {
    return typeid(*this) == typeid(other) && equals(other);
}
```

6. TEMPLATE CLASS FOR ANALYTICS

include/AnalyticsEngine.h, lines 25-26 and 39-47

```
template <typename T = double>
class AnalyticsEngine {

    T computeTotalExpenses(const std::vector<const Transaction*>& transactions)
const {
    T total{};
```

```

        for (const Transaction* t : transactions) {
            if (t->computeImpact()) {
                total += (t->getAmount());
            }
        }
        return total;
    }
}

```

7. SMART POINTERS AND STL CONTAINERS

include/TransactionManager.h, lines 52 and 57

```

std::vector<std::unique_ptr<Transaction>> transactions;

std::vector<std::unique_ptr<Category>> categories;

```

8. MODERN C++: std::expected

include/TransactionManager.h, line 344

```

std::expected<int, DataManager::DataReadWriteError> load(const DataManager&
dataManager);

```

9. EXCEPTION HANDLING

src/GUI.cpp, lines 420-423

```

    } catch (const std::exception& e) {
        QMessageBox::warning(this, "Save Transaction", e.what());
        return;
    }
}

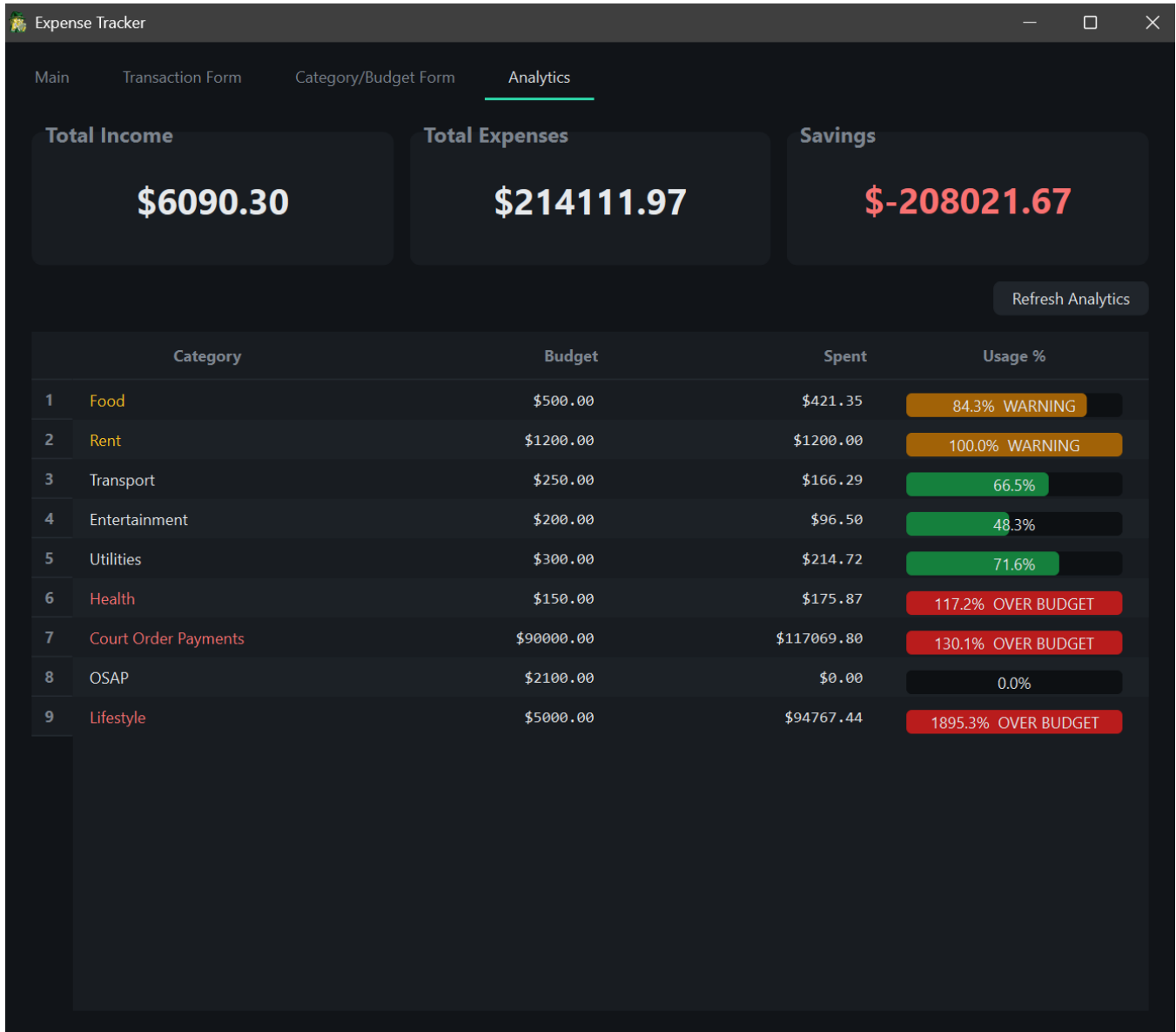
```

Appendix C – Sample Data Files

```
categories.json x GUI.cpp DataManag
1  [
2      {
3          "categoryId": "CAT-0001",
4          "monthlyBudget": 500.0,
5          "name": "Food"
6      },
7      {
8          "categoryId": "CAT-0002",
9          "monthlyBudget": 1200.0,
10         "name": "Rent"
11     },
12     {
13         "categoryId": "CAT-0003",
14         "monthlyBudget": 250.0,
15         "name": "Transport"
16     },
17     {
18         "categoryId": "CAT-0004",
19         "monthlyBudget": 200.0,
20         "name": "Entertainment"
21     }
22 ]
```

```
transactions.json x categories.json GUI.cpp DataManag
1  [
2      {
3          "__type": 1,
4          "amount": 1200.0,
5          "category": "Rent",
6          "date": "2026-07-01",
7          "description": "Monthly rent payment",
8          "paymentMethod": "Bank Transfer",
9          "transactionId": "TRX-0001"
10     },
11     {
12         "__type": 1,
13         "amount": 112.45,
14         "category": "Food",
15         "date": "2026-07-02",
16         "description": "Groceries for the next 1-2 weeks",
17         "paymentMethod": "Debit",
18         "transactionId": "TRX-0002"
19     },
20     {
21         "__type": 1,
22         "amount": 63.49,
23         "category": "Transport",
24         "date": "2026-07-03",
25         "description": "Gas and cigs",
26         "paymentMethod": "Debit",
27         "transactionId": "TRX-0003"
28     }
29 ]
```

Appendix D – GUI Screenshots



Expense Tracker

Main

Transaction Form

Category/Budget Form

Analytics

Category Name:

e.g. Food, Rent, Transport

Monthly Budget:

500.00

Save

Cancel

Category	Monthly Budget
Food	\$500.00
Rent	\$1200.00
Transport	\$250.00
Entertainment	\$200.00
Utilities	\$300.00
Health	\$150.00
Court Order Payments	\$90000.00
OSAP	\$2100.00
Lifestyle	\$5000.00

Expense Tracker

Main

Transaction Form

Category/Budget Form

Analytics

Add Transaction(s)

Edit Transactions

Delete Transaction

Search / Filter

☐ Date: 2026-08-13

Category: Any

Amount: Any

Search

Clear

	ID	Date	Amount	Category	Description	Type	Method/Source
1	TRX-0001	2026-07-01	\$1200.00	Rent	Monthly rent ...	Expense	Bank Transfer
2	TRX-0002	2026-07-02	\$112.45	Food	Groceries for ...	Expense	Debit
3	TRX-0003	2026-07-03	\$63.49	Transport	Gas and cigs	Expense	Debit
4	TRX-0004	2026-07-05	\$21.50	Entertainment	Netflix ...	Expense	Credit
5	TRX-0005	2026-07-08	\$96.42	Utilities	Hydro bill for ...	Expense	Bank Transfer
6	TRX-0006	2026-07-08	\$118.30	Utilities	Internet and ...	Expense	Bank Transfer
7	TRX-0007	2026-07-09	\$98.20	Food	Groceries and ...	Expense	Credit
8	TRX-0008	2026-07-11	\$64.30	Transport	Fuel for the ...	Expense	Credit
9	TRX-0009	2026-07-15	+\$1420.00	Salary	Biweekly payro...	Income	John Defence
10	TRX-0010	2026-07-16	\$44.75	Food	Lunch with the...	Expense	Cash
11	TRX-0011	2026-07-18	\$75.00	Entertainment	Concert tickets...	Expense	Debit
12	TRX-0012	2026-07-19	+\$2100.00	OSAP	Got paid for ...	Income	Bank Transfer
13	TRX-0013	2026-07-19	+\$670.30	Lifestyle	Hard Rock	Income	Casino Chips

Data Files

Transactions File: C:/Users/deirw/CLionProjects/ExpenseTracker/data/transactions.json

Browse...

Categories File: C:/Users/deirw/CLionProjects/ExpenseTracker/data/categories.json

Browse...

Save Data

Load Data

Expense Tracker

Main

Transaction Form

Category/Budget Form

Analytics

Transaction Details

Type:

Expense

Transaction ID:

TRX-0001

Date:

2026-08-13

Amount:

100.50

Category:

e.g. Food, Rent, Transport

Description:

Payment Method:

e.g. Debit, Credit, Cash

Alert Above:

\$1000.00

Save

Cancel